

Chapter 1: Introduction to Compiling

1.1 Compilers

- A compiler is a program that translates source code written in a high-level programming language into a low-level target language (e.g., assembly or machine code).
- It ensures correctness and optimizes performance.
- Compilers differ from interpreters, which execute code line-by-line without producing an intermediate machine code.

1.2 Analysis of the Source Program

- The compilation process involves analyzing the source code to check for correctness and convert it into an intermediate form.
- It consists of three main levels:
 - **Lexical Analysis:** Converts input into tokens.
 - **Syntax Analysis:** Checks the arrangement of tokens based on grammar rules.
 - **Semantic Analysis:** Ensures meaningful code execution.

1.3 The Phases of a Compiler

1. **Lexical Analysis:** Tokenizes the input.
2. **Syntax Analysis:** Parses token sequences.
3. **Semantic Analysis:** Ensures logic correctness.
4. **Intermediate Code Generation:** Converts syntax tree to intermediate representation.
5. **Code Optimization:** Improves execution efficiency.
6. **Code Generation:** Produces final target code.
7. **Symbol Table Management & Error Handling:** Supports all phases.

1.4 Cousins of the Compiler

- **Preprocessors:** Handle macro expansion.
- **Assemblers:** Convert assembly to machine code.
- **Loaders and Linkers:** Manage executable files.

1.5 The Grouping of Phases

- **Front-end (Analysis Phase):** Lexical, Syntax, Semantic analysis.
- **Middle-end:** Intermediate Code Generation and Optimization.
- **Back-end (Synthesis Phase):** Code Optimization and Code Generation.

1.6 Compiler-Construction Tools

- **Lexical Analyzers (Lex, Flex)**
- **Parser Generators (YACC, Bison)**
- **Semantic Analyzers**
- **Intermediate Code Generators**
- **Optimizers**
- **Code Generators**

Chapter 4: Syntax Analysis

4.1 The Role of the Parser

- Ensures syntactic correctness of token sequences.
- Builds a parse tree following a formal grammar.
- Reports errors with meaningful messages.

4.2 Context-Free Grammars (CFGs)

- Defined as $G = (V, T, P, S)$, where:
 - **V:** Set of variables (non-terminals)

- **T**: Set of terminals (tokens)
- **P**: Production rules
- **S**: Start symbol

4.3 Writing a Grammar

- Grammar should be **unambiguous, non-recursive, left-factored, and deterministic**.

4.4 Top-Down Parsing

- Begins from the start symbol, expanding productions recursively.
- Includes **Recursive Descent Parsing** and **Predictive Parsing (LL(1))**.

4.5 Bottom-Up Parsing

- Builds parse tree from leaves to root.
- **Shift-Reduce Parsing, LR(0), SLR(1), LR(1), LALR(1)**.

4.7 LR Parsers

- **LR(0)**: Basic shift-reduce parser.
- **SLR(1)**: Simple LR parser using Follow sets.
- **LR(1)**: Canonical LR parser with lookahead.
- **LALR(1)**: Merges LR(1) states to reduce table size.

4.8 Using Ambiguous Grammars

- An ambiguous grammar allows multiple parse trees for a single string.
- Resolved using precedence and associativity rules.

Chapter 5: Syntax-Directed Translation

5.1 Syntax-Directed Definitions (SDD)

- Associates semantic rules with grammar productions.
- Uses **Synthesized Attributes** (depend on child nodes) and **Inherited Attributes** (depend on parent/siblings).

5.2 Construction of Syntax Trees

- A compact representation of parsing.
- Nodes represent operators; leaves represent operands.

5.3 Bottom-Up Evaluation of S-Attributed Definitions

- Uses **only synthesized attributes**.
- Evaluated using **post-order traversal**.

5.4 L-Attributed Definitions

- Allows both synthesized and inherited attributes but restricts dependencies to left-to-right order.

5.5 Top-Down Translation

- Translates input using **recursive descent parsing**.
- Uses attributes evaluated in preorder traversal.

5.6 Bottom-Up Evaluation of Inherited Attributes

- Inherited attributes are computed **after child nodes** are visited.

5.7 Recursive Evaluators

- Recursively traverse the parse tree for attribute evaluation.

5.10 Analysis of Syntax-Directed Definitions

- Checks for consistency and dependency cycles.
 - Ensures attributes are computed in a valid order.
-

Key Concepts Related to Syntax Analysis

Lexical Analysis

- Converts source code into **tokens**.
- Uses **Regular Expressions and Finite Automata**.
- Implemented using **Lex/Flex**.

Transition Diagrams

- Graphical representation of state transitions in **finite automata**.
- Used for **pattern recognition** in lexical analysis.

Left Recursion

- A grammar is **left-recursive** if a non-terminal appears at the beginning of its production.
- **Example:**
- $A \rightarrow A\alpha \mid \beta$
- Eliminated by transformation:
- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \varepsilon$

Ambiguity

- A grammar is **ambiguous** if a string has more than one parse tree.
- Resolved using **disambiguation rules** or **grammar rewriting**.

Determinism and Non-Determinism

- **Deterministic parsing:** Predicts correct derivations without backtracking (LL(1), LR(1)).
- **Non-Deterministic parsing:** Requires backtracking or lookahead.

Left Factoring

- Removes common prefixes to enable **predictive parsing**.
- **Example:**
- $A \rightarrow \alpha\beta \mid \alpha\gamma$
- Left factored to:
- $A \rightarrow \alpha A'$
- $A' \rightarrow \beta \mid \gamma$

LL(1) Parsing

- **Top-down predictive parsing** with one token lookahead.
- Requires **non-left-recursive** and **left-factored** grammar.

SLR Parsing (Simple LR)

- **Bottom-up shift-reduce parsing** with Follow sets.
- **More powerful than LL(1) but less than LR(1).**

LR(0) Parsing

- **Basic shift-reduce parsing** without lookahead.
- Constructs an **LR parsing table** using **item sets**.

Syntax Directed Translation Schemes

- Syntax-directed translations implement semantic actions within parsing.
- Uses **action symbols** within grammar rules.
- Example (Postfix expression translation):
- $E \rightarrow E_1 + T \{ \text{print}(' + '); \}$